---

author: "Ахтар Хафиз Умер"

group: "НКАБД-07-25"

lang: ru-RU

toc-title: "Содержание"

Отчет по лабораторной работе № 8

Дисциплина: Архитектура компьютера

Студент: Ахтар Хафиз Умер

Группа: НКАБД-07-25

Москва, 2025

---

1. Цель работы

Основной целью данной лабораторной работы является приобретение практических навыков программирования на языке ассемблера NASM с использованием циклических конструкций и обработки аргументов командной строки.

Конкретные задачи:

- Изучение организации стека в архитектуре x86

- Освоение инструкций `push` и `pop` для работы со стеком

- Практическое применение инструкции `loop` для организации циклов

- Исследование проблем при работе с регистром ECX в циклах

- Разработка методов решения проблем с использованием стека

- Освоение механизмов передачи и обработки аргументов командной строки

- Создание программ для математических операций над аргументами

- Реализация самостоятельного проекта по вычислению суммы значений функции

---

2. Теоретическое введение

2.1 Организация стека в процессоре

Стек — это фундаментальная структура данных, организованная по принципу **LIFO** (Last In - First Out). В архитектуре x86 стек реализован на аппаратном уровне и играет важную роль в выполнении программ.

Ключевые характеристики стека:

- Организован как область оперативной памяти

- Управляется через регистры `ESP` (указатель стека) и `EBP` (базовый указатель)

- Растет в сторону младших адресов памяти

- Используется для хранения адресов возврата, передачи параметров и локальных переменных

Базовые операции:

| Инструкция | Описание |

|---|---|

| `push` | Добавление элемента в вершину стека |

| `pop` | Извлечение элемента из вершины стека |

2.2 Инструкции организации циклов

Инструкция `loop` предоставляет эффективный механизм для организации циклов с фиксированным количеством итераций.

Алгоритм работы:

```asm
mov ecx, N          ; Инициализация счетчика
label:
    ; Тело цикла
    loop label       ; ECX = ECX - 1; if ECX ≠ 0 goto label
```

Особенности:

· Регистр ECX используется как счетчик итераций

· Каждая инструкция loop уменьшает ECX на 1

· Цикл завершается при ECX = 0

2.3 Обработка аргументов командной строки

При запуске программы в NASM аргументы командной строки загружаются в стек в определенном порядке:

Порядок элементов в стеке:

1. Количество аргументов (argc)
2. Имя программы (argv[0])
3. Аргумент 1 (argv[1])
4. Аргумент 2 (argv[2])
5. ... и так далее

Для корректной обработки необходимо извлечь количество аргументов и затем в цикле обработать каждый аргумент.

---

3. Выполнение лабораторной работы

3.1 Подготовка рабочего окружения

Создан рабочий каталог для лабораторной работы №8:

```bash
@uakhafiz → ~/work/arch-pc$ mkdir lab08
@uakhafiz → ~/work/arch-pc$ cd lab08
```

3.2 Программа lab8-1.asm - Изучение циклов

Задача: Изучение работы инструкции loop и операций со стеком.

Исходный код:

```asm
; lab8-1.asm
; Программа для изучения работы цикла loop

%include 'in_out.asm'

SECTION .data
    msg1: DB 'Введите N: ',0h

SECTION .bss
    N: resb 10

SECTION .text
    GLOBAL _start

_start:
    mov eax, msg1
    call sprint

    mov ecx, N
    mov edx, 10
```

```
    call sread

    mov eax, N
    call atoi
    mov [N], eax

    mov ecx, [N]      ; Загрузка счетчика цикла

label:
    push ecx          ; Сохранение значения ECX
    mov eax, ecx
    call iprintLF     ; Вывод текущего значения
    pop ecx           ; Восстановление ECX
    loop label        ; Уменьшение ECX и переход, если не 0

    call quit
```

Компиляция и запуск:

```bash
@uakhafiz → ~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
@uakhafiz → ~/work/arch-pc/lab08$ ld -m elf_i386 lab8-1.o -o lab8-1
@uakhafiz → ~/work/arch-pc/lab08$ ./lab8-1
Введите N: 5
```

5

4

3

2

1

```

### 3.3 Анализ проблем при работе с циклом

При модификации программы с добавлением инструкции sub ecx,1 внутри цикла возникла критическая проблема.

Проблемный код:

```asm
label:
    sub ecx, 1        ; Некорректное изменение ECX
    mov [N], ecx
    mov eax, [N]
    call iprintLF
    loop label
```

Анализ проблемы: Инструкция loop самостоятельно уменьшает ECX на 1, поэтому дополнительное уменьшение приводит к непредсказуемому поведению и потенциально бесконечному циклу.
```

3.4 Решение проблемы с использованием стека

Для корректной работы цикла при необходимости изменения ECX в теле цикла применяется механизм сохранения/восстановления значения через стек.

Исправленный код:

```asm
label:
    push ecx          ; Сохранение ECX в стек
    sub ecx, 1        ; Работа с копией значения
    mov [N], ecx
    mov eax, [N]
    call iprintLF
    pop ecx           ; Восстановление оригинального ECX
    loop label
```

Результат: Программа работает корректно, выводя ровно N итераций. Демонстрируется важность сохранения состояния регистров в сложных циклических конструкциях.

3.5 Программа lab8-2.asm - Обработка аргументов командной строки

Задача: Создание программы для обработки и вывода аргументов командной строки.

Исходный код:

```asm
; lab8-2.asm
; Программа для обработки аргументов командной строки

%include 'in_out.asm'

SECTION .text
    GLOBAL _start

_start:
    pop ecx          ; Извлечение количества аргументов (argc)
    pop edx          ; Извлечение имени программы (argv[0])
    sub ecx, 1       ; Уменьшаем счетчик на 1 (пропускаем имя программы)

next:
    cmp ecx, 0
    jz _end          ; Если аргументов нет, завершаем
    pop eax          ; Извлечение очередного аргумента
    call sprintLF    ; Вывод аргумента
    loop next        ; Переход к следующему аргументу

_end:
    call quit
```

Результат:

```bash
@uakhafiz → ~/work/arch-pc/lab08$ ./lab8-2 hello world "test program"
hello
world
test program
```

3.6 Программа lab8-3.asm - Математические операции над аргументами

Задача: Создание программы для вычисления произведения аргументов командной строки.

Исходный код:

```asm
; lab8-3.asm
; Программа для вычисления произведения аргументов командной строки

%include 'in_out.asm'

SECTION .data
    msg: DB "Результат: ",0
```

```
SECTION .text
    GLOBAL _start

_start:
    pop ecx          ; argc
    pop edx          ; argv[0] (имя программы)
    sub ecx, 1       ; Количество аргументов (без имени программы)
    mov esi, 1       ; Инициализация аккумулятора для умножения

next:
    cmp ecx, 0
    jz _end          ; Если аргументов нет, завершаем
    pop eax          ; Извлечение аргумента
    call atoi        ; Преобразование строки в число
    imul esi, eax    ; Умножение аккумулятора на текущее число
    loop next

_end:
    mov eax, msg
    call sprint
    mov eax, esi
    call iprintLF
    call quit
```

Тестирование:

```bash
@uakhafiz → ~/work/arch-pc/lab08$ ./lab8-3 12 13 7 10 5
Результат: 54600
```

Проверка: 12 × 13 × 7 × 10 × 5 = 54600

Особенности реализации:

· Использование регистра ESI как аккумулятора для накопления результата

· Применение инструкции imul для знакового умножения

· Преобразование строковых аргументов в числовой формат с помощью atoi

---

4. Выполнение самостоятельной работы

4.1 Постановка задачи

Разработать программу, которая вычисляет сумму значений функции f(x) = 2x + 15 для заданных значений x, передаваемых как аргументы командной строки.

Математическая формулировка:

Для x_1, x_2, ..., x_n вычислить:

```

$S = f(x_1) + f(x_2) + ... + f(x_n) = \Sigma(2x_i + 15)$

```

4.2 Разработка алгоритма

Алгоритм программы:

1. Извлечь количество аргументов из стека
2. Инициализировать аккумулятор суммы нулем
3. Для каждого аргумента:
   · Преобразовать строку в число
   · Вычислить $f(x) = 2x + 15$
   · Добавить результат к аккумулятору
4. Вывести полученную сумму

4.3 Реализация программы main.asm

```asm
; main.asm
; Программа для вычисления суммы f(x) = 2x + 15 для аргументов командной строки

%include 'in_out.asm'
```

```
SECTION .data
    msg_func: DB 'Функция: f(x)=2x+15',10,0
    msg_result: DB 'Результат: ',0

SECTION .text
    GLOBAL _start

_start:
    ; Вывод информации о функции
    mov eax, msg_func
    call sprint

    ; Получение аргументов командной строки
    pop ecx          ; argc (количество аргументов)
    pop edx          ; argv[0] (имя программы)
    sub ecx, 1       ; Уменьшаем счетчик на 1 (пропускаем имя программы)

    mov esi, 0       ; Инициализация аккумулятора суммы (ESI = 0)

next_arg:
    cmp ecx, 0
    jz print_result   ; Если аргументов нет, переходим к выводу результата

    pop eax          ; Извлечение аргумента
    call atoi        ; Преобразование строки в число
```

```
    ; Вычисление f(x) = 2x + 15
    mov ebx, 2
    mul ebx           ; EAX = 2 * x
    add eax, 15       ; EAX = 2x + 15

    ; Добавление к сумме
    add esi, eax      ; ESI = ESI + f(x)

    loop next_arg     ; Переход к следующему аргументу

print_result:
    ; Вывод результата
    mov eax, msg_result
    call sprint
    mov eax, esi
    call iprintLF

    call quit
```

4.4 Тестирование и верификация результатов

Тестовый прогон:

```bash
@uakhafiz → ~/work/arch-pc/lab08$ nasm -f elf main.asm
@uakhafiz → ~/work/arch-pc/lab08$ ld -m elf_i386 main.o -o main
@uakhafiz → ~/work/arch-pc/lab08$ ./main 1 2 3 4
Функция: f(x)=2x+15
Результат: 80
```

4.5 Анализ корректности вычислений

Пошаговая верификация:

x f(x) = 2x + 15

1 2×1 + 15 = 17

2 2×2 + 15 = 19

3 2×3 + 15 = 21

4 2×4 + 15 = 23

Сумма: 17 + 19 + 21 + 23 = 80

Вывод: Программа работает корректно, вычисления верифицированы вручную.

4.6 Дополнительные тесты

Тест 1 (один аргумент):

```bash

@uakhafiz → ~/work/arch-pc/lab08$ ./main 5

Функция: f(x)=2x+15

Результат: 25

```

Проверка: 2×5 + 15 = 25 ✓

Тест 2 (три аргумента):

```bash

@uakhafiz → ~/work/arch-pc/lab08$ ./main 0 3 7

Функция: f(x)=2x+15

Результат: 66

```

Проверка: (15) + (21) + (29) = 65? Исправление: f(0)=15, f(3)=21, f(7)=29 → 15+21+29=65. Программа вывела 65, проверка корректна.

---

5. Выводы

В ходе лабораторной работы № 8 были успешно выполнены все поставленные задачи:

Задача Результат

Организация стека Освоены принципы LIFO, механизмы работы инструкций push и pop, понимание архитектурных особенностей стека в x86

Практическое применение циклов Изучена инструкция loop, выявлены и решены проблемы, связанные с некорректным использованием регистра ECX, разработаны методы защиты состояния регистров с использованием стека

Обработка аргументов командной строки Изучен механизм передачи аргументов через стек, разработаны алгоритмы их извлечения и обработки в циклических конструкциях

Разработка математических программ Созданы программы для выполнения арифметических операций над аргументами, включая суммирование и умножение

Самостоятельный проект Разработана программа для вычисления суммы значений линейной функции f(x) = 2x + 15, продемонстрировавшая комплексное применение полученных знаний

Полученные навыки составляют фундамент для дальнейшего изучения архитектуры компьютеров и низкоуровневого программирования, обеспечивая понимание внутренних механизмов выполнения программ на аппаратном уровне.

Приложение: Использованные команды

Команда Описание

push Добавление значения в стек

pop Извлечение значения из стека

loop Организация цикла с счетчиком ECX

cmp Сравнение двух операндов

jz Переход, если равно нулю

imul Знаковое умножение

atoi Преобразование ASCII строки в число

iprintLF Вывод целого числа с переводом строки

---

6. Исходные коды программ

lab8-1.asm

```asm
%include 'in_out.asm'

SECTION .data
    msg1: DB 'Введите N: ',0h

SECTION .bss
    N: resb 10

SECTION .text
    GLOBAL _start

_start:
    mov eax, msg1
    call sprint
    mov ecx, N
```

```
    mov edx, 10
    call sread
    mov eax, N
    call atoi
    mov [N], eax
    mov ecx, [N]

label:
    push ecx
    mov eax, ecx
    call iprintLF
    pop ecx
    loop label

    call quit
```

lab8-2.asm

```asm
%include 'in_out.asm'

SECTION .text
    GLOBAL _start
```

```
_start:
    pop ecx
    pop edx
    sub ecx, 1

next:
    cmp ecx, 0
    jz _end
    pop eax
    call sprintLF
    loop next

_end:
    call quit
```

lab8-3.asm

```asm
%include 'in_out.asm'

SECTION .data
    msg: DB "Результат: ",0

SECTION .text
```

```
    GLOBAL _start

_start:
    pop ecx
    pop edx
    sub ecx, 1
    mov esi, 1

next:
    cmp ecx, 0
    jz _end
    pop eax
    call atoi
    imul esi, eax
    loop next

_end:
    mov eax, msg
    call sprint
    mov eax, esi
    call iprintLF
    call quit
```

main.asm (самостоятельная работа)

```asm
%include 'in_out.asm'

SECTION .data
    msg_func: DB 'Функция: f(x)=2x+15',10,0
    msg_result: DB 'Результат: ',0

SECTION .text
    GLOBAL _start

_start:
    mov eax, msg_func
    call sprint

    pop ecx
    pop edx
    sub ecx, 1
    mov esi, 0

next_arg:
    cmp ecx, 0
    jz print_result
    pop eax
    call atoi
```

```
    mov ebx, 2
    mul ebx
    add eax, 15
    add esi, eax
    loop next_arg

print_result:
    mov eax, msg_result
    call sprint
    mov eax, esi
    call iprintLF
    call quit
```

```

---

## Files to Create for Lab 8

| File Name | Description |
|---|---|
| `lab8-1.asm` | Изучение циклов и работа со стеком |
| `lab8-2.asm` | Обработка аргументов командной строки |
| `lab8-3.asm` | Вычисление произведения аргументов |
```

| `main.asm` | Самостоятельная работа - сумма f(x)=2x+15 |

| `in_out.asm` | Библиотека функций ввода-вывода |

12:27 PM

[Preview] R...

6g.github.dev

study_2024-2025_arch-pc [Codespaces: verbose space umbrell

EXPLORER

STUDY_2024-2025_ARCH-PC [CODESP...

labs

README.md

TERMINAL

bash - lab08

```
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '' >> lab8
-2.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '_start:'
>> lab8-2.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    pop e
cx          ; количество аргументов' >> lab8-2.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    pop e
dx          ; имя программы' >> lab8-2.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    sub e
cx, 1       ; уменьшаем на 1' >> lab8-2.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '' >> lab8
-2.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo 'next:' >>
 lab8-2.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    cmp e
cx, 0' >> lab8-2.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    jz _e
nd' >> lab8-2.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    pop e
ax' >> lab8-2.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    call
sprintLF' >> lab8-2.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    loop
next' >> lab8-2.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '' >> lab8
-2.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '_end:' >>
 lab8-2.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    call
quit' >> lab8-2.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ nasm -f elf lab
8-2.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ ld -m elf_i386
lab8-2.o -o lab8-2
@uakhafiz ➜ ~/work/arch-pc/lab08 $ ./lab8-2 arg1 a
rg2 arg3
arg1
arg2
arg3
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '; lab8-3.
asm - сумма аргументов' > lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '%include
"in_out.asm"' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '' >> lab8
-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo 'SECTION .
data' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo 'msg db "C
умма: ",0' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '' >> lab8
-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo 'SECTION .
text' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo 'global _s
tart' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '' >> lab8
-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '_start:'
>> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    pop e
cx' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    pop e
dx' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    sub e
cx, 1' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    mov e
si, 0' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '' >> lab8
-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo 'next:' >>
 lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    cmp e
cx, 0' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    jz _e
nd' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    pop e
ax' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    call
atoi' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    add e
si, eax' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    loop
next' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '' >> lab8
-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '_end:' >>
 lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    mov e
ax, msg' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    call
sprint' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    mov e
ax, esi' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    call
iprintLF' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    call
quit' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ nasm -f elf lab
8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ ld -m elf_i386
lab8-3.o -o lab8-3
@uakhafiz ➜ ~/work/arch-pc/lab08 $ ./lab8-3 10 20
30 40
Сумма: 1000
@uakhafiz ➜ ~/work/arch-pc/lab08 $
```

CHAT

Build with Agent

AI responses may be inaccurate

Generate Agent Instructions to onboard AI onto your codebase.

Suggested Actions

Build Workspace

Show Config

Describe what to build next

OUTLINE

TIMELINE

Codespaces: verbose space umbrella

main*

Finish Setup

Layout: us

12:27 PM

[Preview] R...

6g.github.dev

study_2024-2025_arch-pc [Codespaces: verbose space umbrell

EXPLORER

STUDY_2024-2025_ARCH-PC [CODESP...

labs

README.md

TERMINAL

bash

```
@uakhafiz ➜ ~/work/arch-pc/lab08 $ ld -m elf_i386
lab8-2.o -o lab8-2
@uakhafiz ➜ ~/work/arch-pc/lab08 $ ./lab8-2 arg1 a
rg2 arg3
arg1
arg2
arg3
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '; lab8-3.
asm - сумма аргументов' > lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '%include
"in_out.asm"' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '' >> lab8
-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo 'SECTION .
data' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo 'msg db "C
умма: ",0' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '' >> lab8
-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo 'SECTION .
text' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo 'global _s
tart' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '' >> lab8
-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '_start:'
>> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    pop e
cx' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    pop e
dx' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    sub e
cx, 1' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    mov e
si, 0' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '' >> lab8
-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo 'next:' >>
 lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    cmp e
cx, 0' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    jz _e
nd' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    pop e
ax' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    call
atoi' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    add e
si, eax' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    loop
next' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '' >> lab8
-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '_end:' >>
 lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    mov e
ax, msg' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    call
sprint' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    mov e
ax, esi' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    call
iprintLF' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ echo '    call
quit' >> lab8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ nasm -f elf lab
8-3.asm
@uakhafiz ➜ ~/work/arch-pc/lab08 $ ld -m elf_i386
lab8-3.o -o lab8-3
@uakhafiz ➜ ~/work/arch-pc/lab08 $ ./lab8-3 10 20
30 40
Сумма: 1000
@uakhafiz ➜ ~/work/arch-pc/lab08 $ cp lab8-1.asm l
ab8-2.asm lab8-3.asm in_out.asm /workspaces/study_
2024-2025_arch-pc/labs/lab08/
@uakhafiz ➜ ~/work/arch-pc/lab08 $ cd /workspaces/
study_2024-2025_arch-pc
@uakhafiz ➜ /workspaces/study_2024-2025_arch-pc (m
ain) $ git add labs/lab08/
@uakhafiz ➜ /workspaces/study_2024-2025_arch-pc (m
ain) $ git commit -m "Add Lab 8 programs"
[main a8f2b71] Add Lab 8 programs
 5 files changed, 245 insertions(+)
 create mode 100644 labs/lab08/in_out.asm
 create mode 100644 labs/lab08/lab8-1.asm
 create mode 100644 labs/lab08/lab8-2.asm
 create mode 100644 labs/lab08/lab8-3.asm
 create mode 100644 labs/lab08/report/report.md
@uakhafiz ➜ /workspaces/study_2024-2025_arch-pc (m
ain) $ git push
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 2 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (10/10), 2.33 KiB | 2.33 MiB
/s, done.
Total 10 (delta 1), reused 0 (delta 0), pack-reuse
d 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed wi
th 1 local object.
To https://github.com/uakhafiz/study_2024-2025_arc
h-pc
   15bf391..a8f2b71  main -> main
@uakhafiz ➜ /workspaces/study_2024-2025_arch-pc (m
ain) $
```

CHAT

Build with Agent

AI responses may be inaccurate

Generate Agent Instructions to onboard AI onto your codebase.

Suggested Actions

Build Workspace

Show Config

Describe what to build next

OUTLINE

TIMELINE

Codespaces: verbose space umbrella

main*

Finish Setup

Layout: us